

CLASS 2 - SOLANA INDIA BOOTCAMP

RUST ADVANCE

Topics

1. Box

2. Deref

3. Rc

4. RefCell

5. Arc

6. Mutex

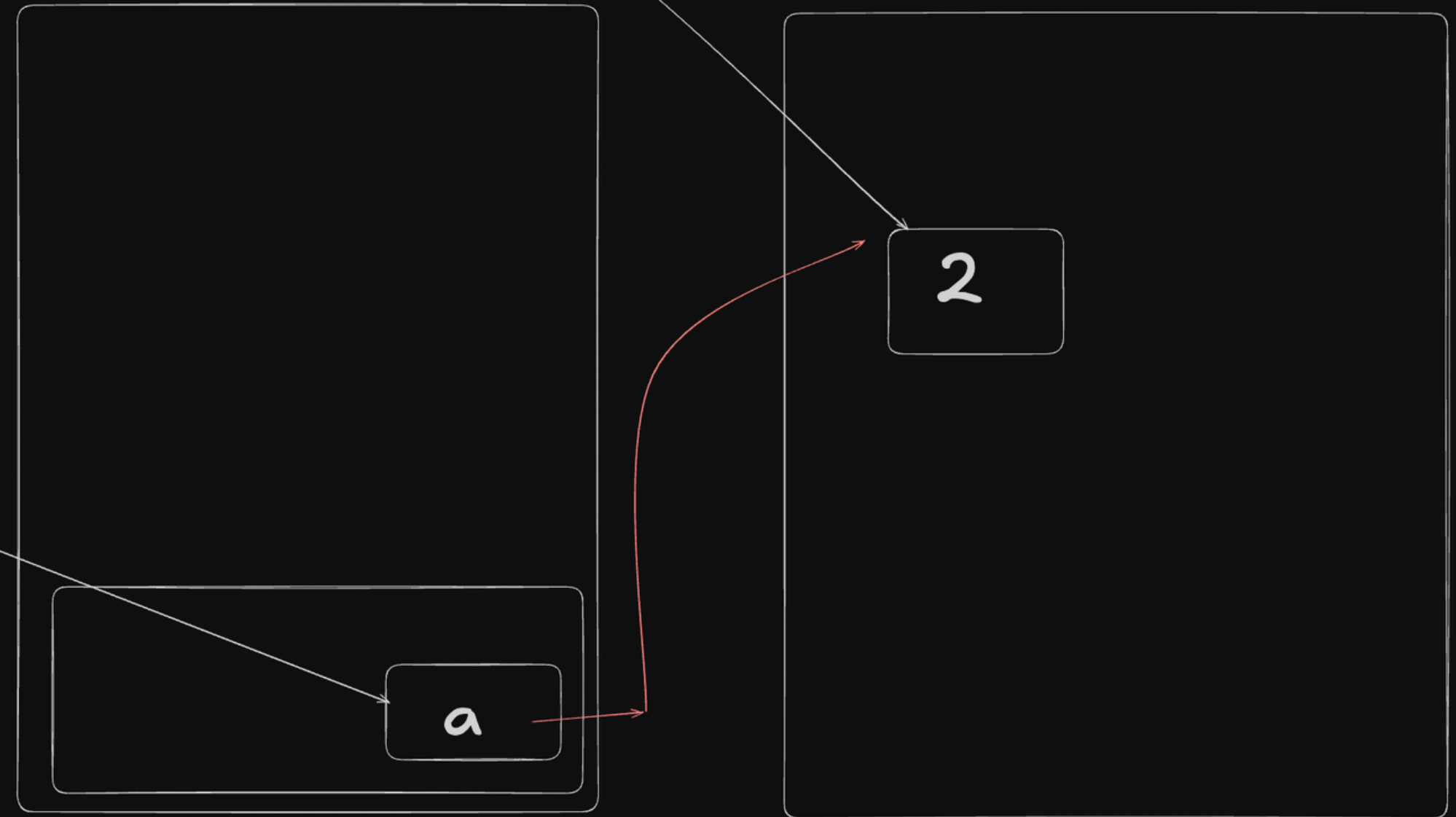
7. mpsc

Box

```
fn main() {  
    let a: Box<i32> = Box::new(2);  
    println!("{:p}", a); // heap value  
    println!("{:p}", &a); // stack value  
}
```

stack

heap



Box

Q - How would you build a linked list in rust?

A linked list is a linear data structure consisting of nodes, where each node contains data and a pointer (link) to the next node in the sequence. Unlike arrays, linked list elements are not stored in contiguous memory locations, allowing for efficient insertion and deletion of nodes. It starts with a "head" node and ends at a "tail" node that points to NULL.  Reddit +3

Box

Q - How would you build a linked list in rust?

```
1  | ⌘L to chat, ⌘K to generate
2
3  #[derive(Debug)]
4  1 implementation
5  struct LinkedList {
6      value: i32,
7      next: Option<LinkedList>
8  }
9
10 ▶ Run | ⌘ Debug
11 fn main() {
12     let ll: LinkedList = LinkedList {
13         value: 1,
14         next: None
15     };
16
17     print!("{:?}", ll);
18 }
```

```
recursive type `LinkedList` has infinite size rustc(Click for full compiler diagnostic)
main.rs(6, 18): recursive without indirection
main.rs(6, 18): insert some indirection (e.g., a `Box`, `Rc`, or `&`) to break the cycle: `Box<`, `>`
```


Box

Q - How would you build a linked list in rust?

```
#[derive(Debug)]
1 implementation
struct LinkedList {
    value: i32,
    next: Option<Box<LinkedList>>
}

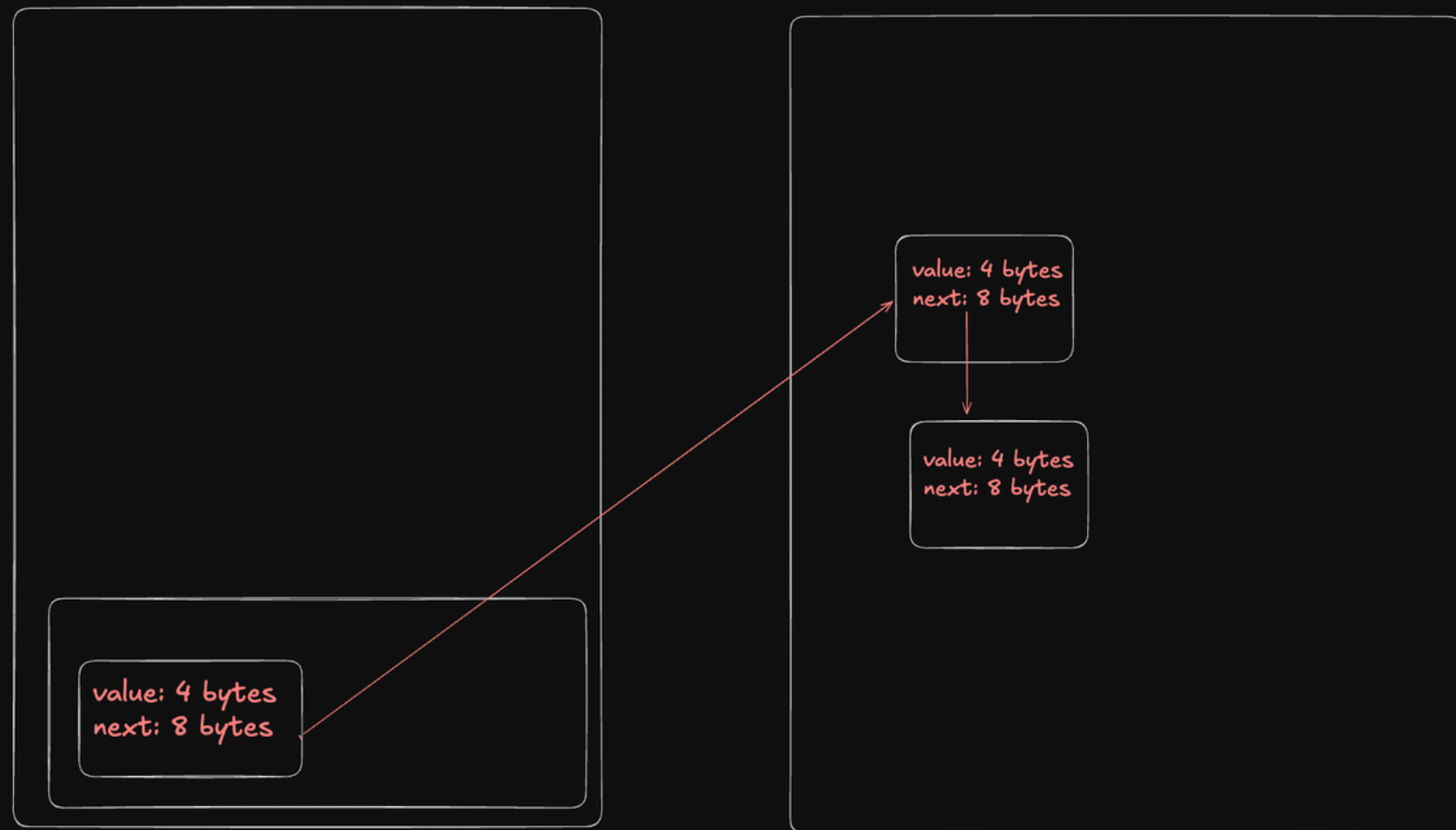
► Run | ⌕ Debug
fn main() {
    let ll: LinkedList = LinkedList {
        value: 1,
        next: Some(Box::new(LinkedList {
            value: 2,
            next: None
        })),
    };

    print!("{:?}", ll);
}
```

⌘L to chat, ⌘K to generate

stack

heap



Topics

1. Box

2. Deref

3. Rc

4. RefCell

5. Arc

6. Mutex

7. mpsc

Deref

The Deref trait lets you control what happens when you use the `*` operator on a type. Rust uses deref coercion to automatically convert `&Box` to `&T`, making smart pointers ergonomic.

Deref (1. wrapper structs)

```
use std::ops::Deref;
```

```
1 implementation
```

```
struct Wrapper<T>(T);
```

```
impl<T> Deref for Wrapper<T> {
```

```
    type Target = T;
```

```
    fn deref(&self) -> &Self::Target {
```

```
        &self.0      // returns ref to inner value
```

```
    }
```

```
}
```

► Run | ⚙ Debug

```
fn main() {
```

```
    let w: Wrapper<String> = Wrapper(String::from("hello"));
```

```
    println!("{}", *w);
```

```
}
```

Deref (2. Box deref coercion)

► Run | ⚙ Debug

```
fn main() {  
    let b: Box<i32> = Box::new(2);  
    let c: Box<i32> = Box::new(3);  
    let ans: i32 = *b + *c;  
  
    print!("{}", ans);  
}
```

Topics

1. Box

2. Deref

3. Rc

4. RefCell

5. Arc

6. Mutex

7. mpsc

Rc (reference counting)

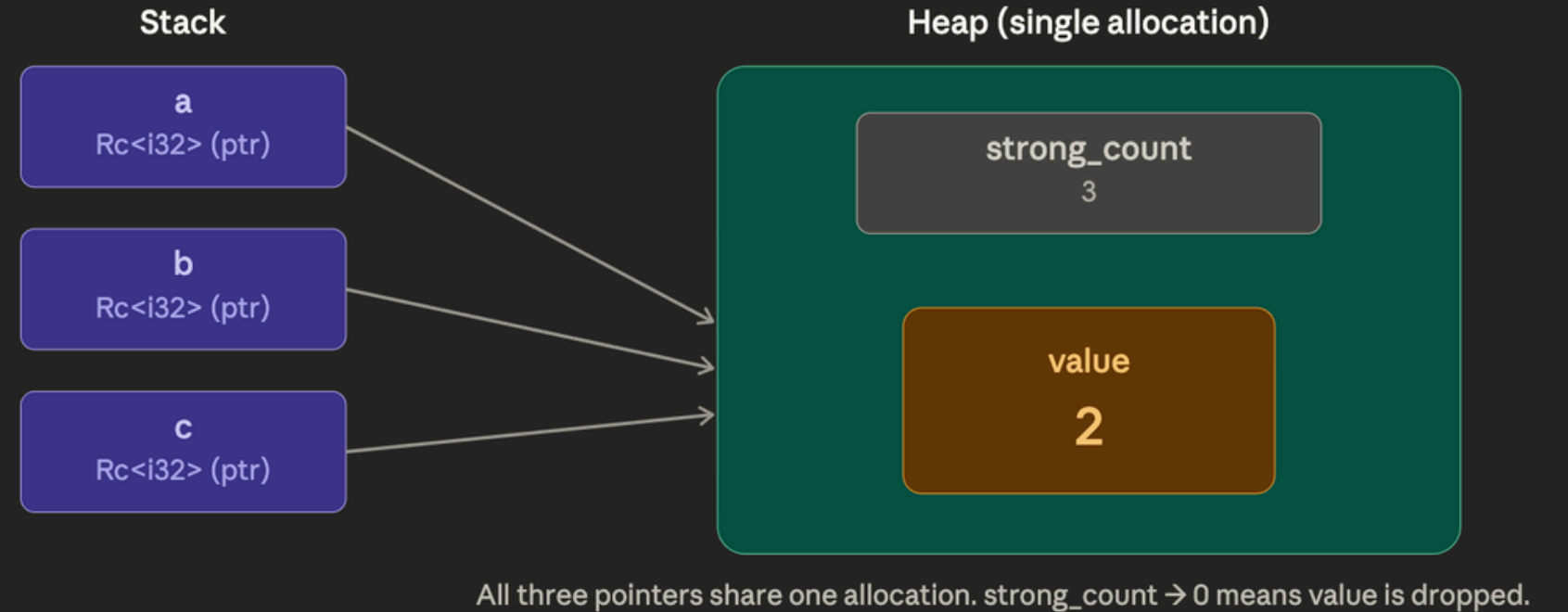
Rc (Reference Counted) allows multiple parts of your program to own the same data. The data is kept alive as long as any Rc pointing to it exists. Single-threaded only.

Rc (reference counting)

```
use std::rc::Rc;
```

► Run | ⚙ Debug

```
fn main() {  
    let a: Rc<i32> = Rc::new(2);  
    let b: Rc<i32> = a.clone();  
    let c: Rc<i32> = a.clone();  
    print!("{}", a);  
    print!("{}", b);  
    print!("{}", c);  
}
```



Topics

1. Box

2. Deref

3. Rc

4. RefCell

5. Arc

6. Mutex

7. mpsc

RefCell

RefCell moves borrow checking from compile time to runtime. It lets you hold mutable references even in contexts where the borrow checker would normally refuse — at the cost of a potential panic.

RefCell

What is the issue here?

```
use std::rc::Rc;
```

► Run | ⚙ Debug

```
fn main() {  
    let mut a: String = String::from("hi there");  
    let b: String = a;  
    a.push_str(&String::from("harkirat"));  
  
    print!("{}", a);  
}
```

⌘L to chat, ⌘K to generate

RefCell

What is the issue here?

```
borrow of moved value: `a`  
value borrowed here after move rustc(Click for full compiler diagnostic)  
  
main.rs(5, 13): value moved here  
  
main.rs(4, 9): move occurs because `a` has type `String`, which does not implement the `Copy` trait  
  
main.rs(5, 14): consider cloning the value if the performance cost is acceptable: `.clone()`
```

```
use std::rc::Rc;
```

► Run | ⚙ Debug

```
fn main() {  
    let mut a: String = String::from("hi there");  
    let b: String = a;  
    a.push_str(&String::from("harkirat"));  
  
    print!("{}", a);  
}
```

⌘L to chat, ⌘K to generate

Borrow checker complains

RefCell moves borrow checking from compile time to runtime.

RefCell (Use case 1 - interior mutability)

```
use std::cell::RefCell;
```

► Run | ⚙ Debug

```
fn main() {  
    let a: RefCell<String> = RefCell::new(String::from("hi there"));  
    a.borrow_mut().push_str(&String::from("harkirat")); // can mutate an immutable variable  
  
    print!("{}", a.borrow());  
}
```

⌘L to chat, ⌘K to generate

RefCell (Use case 2)

```
use std::cell::RefCell;  
// This does NOT compile
```

1 implementation

```
struct Graph {  
    edges: Vec<String>,  
}
```

cannot borrow `\*self` as mutable because it is also borrowed as immutable
mutable borrow occurs here [rustc\(Click for full compiler diagnostic\)](#)

```
impl Graph {  
    fn process(&mut self) {  
        let first: &String = &self.edges[0];  
        self.add_edge("new".to_string());  
        println!("{}", first);  
    }
```

```
    fn add_edge(&mut self, e: String) {  
        self.edges.push(e);  
    }  
}
```

Whats the error?

RefCell (Use case 2)

```
use std::cell::RefCell;
// This does NOT compile
1 implementation
struct Graph {
    edges: Vec<String>,
}

impl Graph {
    fn process(&mut self) {
        let first: &String = &self.edges[0];
        self.add_edge("new".to_string());
        println!("{}", first);
    }

    fn add_edge(&mut self, e: String) {
        self.edges.push(e);
    }
}
```

```
use std::cell::RefCell;
// This does NOT compile
1 implementation
struct Graph {
    edges: RefCell<Vec<String>>,
}

impl Graph {
    fn process(&self) {
        let first: &String = &self.edges.borrow()[0];
        self.add_edge("new".to_string());
        println!("{}", first);
    }

    fn add_edge(&self, e: String) {
        self.edges.borrow_mut().push(e);
    }
}

▶ Run | ⌘ Debug
fn main() {
    let g: Graph = Graph {
        edges: RefCell::new(vec![String::from("first")])
    };

    g.process();
}
```

RefCell (Use case 2)

Q1 - Will this compile?

```
use std::cell::RefCell;
// This does NOT compile
1 implementation
struct Graph {
    edges: Vec<String>,
}

impl Graph {
    fn process(&mut self) {
        let first: &String = &self.edges[0];
        self.add_edge("new".to_string());
        println!("{}", first);
    }

    fn add_edge(&mut self, e: String) {
        self.edges.push(e);
    }
}
```

```
use std::cell::RefCell;
// This does NOT compile
1 implementation
struct Graph {
    edges: RefCell<Vec<String>>,
}

impl Graph {
    fn process(&self) {
        let first: &String = &self.edges.borrow()[0];
        self.add_edge("new".to_string());
        println!("{}", first);
    }

    fn add_edge(&self, e: String) {
        self.edges.borrow_mut().push(e);
    }
}

▶ Run | ⌘ Debug
fn main() {
    let g: Graph = Graph {
        edges: RefCell::new(vec![String::from("first")])
    };

    g.process();
}
```


RefCell (Use case 2)

Q1 - Will this compile? - yes

```
use std::cell::RefCell;
// This does NOT compile
1 implementation
struct Graph {
    edges: Vec<String>,
}

impl Graph {
    fn process(&mut self) {
        let first: &String = &self.edges[0];
        self.add_edge("new".to_string());
        println!("{}", first);
    }

    fn add_edge(&mut self, e: String) {
        self.edges.push(e);
    }
}
```

```
use std::cell::RefCell;
// This does NOT compile
1 implementation
struct Graph {
    edges: RefCell<Vec<String>>,
}

impl Graph {
    fn process(&self) {
        let first: &String = &self.edges.borrow()[0];
        self.add_edge("new".to_string());
        println!("{}", first);
    }

    fn add_edge(&self, e: String) {
        self.edges.borrow_mut().push(e);
    }
}

▶ Run | ⌘ Debug
fn main() {
    let g: Graph = Graph {
        edges: RefCell::new(vec![String::from("first")])
    };

    g.process();
}
```


RefCell (Use case 2)

Q2 - Will this run?

```
use std::cell::RefCell;
// This does NOT compile
1 implementation
struct Graph {
    edges: Vec<String>,
}

impl Graph {
    fn process(&mut self) {
        let first: &String = &self.edges[0];
        self.add_edge("new".to_string());
        println!("{}", first);
    }

    fn add_edge(&mut self, e: String) {
        self.edges.push(e);
    }
}
```

```
use std::cell::RefCell;
// This does NOT compile
1 implementation
struct Graph {
    edges: RefCell<Vec<String>>,
}

impl Graph {
    fn process(&self) {
        let first: &String = &self.edges.borrow()[0];
        self.add_edge("new".to_string());
        println!("{}", first);
    }

    fn add_edge(&self, e: String) {
        self.edges.borrow_mut().push(e);
    }
}

▶ Run | ⌘ Debug
fn main() {
    let g: Graph = Graph {
        edges: RefCell::new(vec![String::from("first")])
    };

    g.process();
}
```

RefCell (Use case 2)

Q2 - Will this run?

```
use std::cell::RefCell;
// This does NOT compile
1 implementation
struct Graph {
    edges: Vec<String>,
}

impl Graph {
    fn process(&mut self) {
        let first: &String = &self.edges[0];
        self.add_edge("new".to_string());
        println!("{}", first);
    }

    fn add_edge(&mut self, e: String) {
        self.edges.push(e);
    }
}
```

```
use std::cell::RefCell;
// This does NOT compile
1 implementation
struct Graph {
    edges: RefCell<Vec<String>>,
}

impl Graph {
    fn process(&self) {
        let first: &String = &self.edges.borrow()[0];
        self.add_edge("new".to_string());
        println!("{}", first);
    }

    fn add_edge(&self, e: String) {
        self.edges.borrow_mut().push(e);
    }
}

fn main() {
    let g: Graph = Graph {
        edges: RefCell::new(vec![String::from("first")])
    };
    g.process();
}
```

note: run with `RUST\_BACKTRACE=1` environment variable to display a backtrace

```
rust-project cargo run
Compiling scheduler v0.1.0 (/Users/harkirat/Projects/rust-project)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.41s
Running `target/debug/scheduler`

thread 'main' (1497136) panicked at src/main.rs:15:20:
RefCell already borrowed
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```


So whats the point of refcell?

It just moved the error from compile time to runtime. What benefit does it provide?

So whats the point of refcell?

- none really. it lets you write code in a way that would've failed during compilation but might succeed during runtime

There is certain code you cant write/wont compile w/o using Rc + RefCell

The real benefit comes with Rc and RefCel are used together.

Q: You have a Logger that both a Database and a Server component hold. When either logs something, the other should see it when they read the log.

```
0 implementations  
struct Logger {  
    logs: Vec<String>,  
}
```

```
0 implementations  
struct Database {  
    logger: Logger,  
}
```

```
0 implementations  
struct Server {  
    logger: Logger,  
}
```

| ⌘L to chat ⌘K to generate

Q: You have a Logger that both a Database and a Server component hold. When either logs something, the other should see it when they read the log.

```
0 implementations
✓ struct Logger {
    logs: Vec<String>,
}

0 implementations
✓ struct Database {
    logger: Logger,
}

0 implementations
✓ struct Server {
    logger: Logger,
}

► Run | ⚙ Debug
fn main() {
    let logger: Logger = Logger { logs: vec![] };

    let mut db: Database = Database { logger };

    let mut server: Server = Server { logger }; // use of moved value

    db.logger.logs.push("db connected".to_string());
    server.logger.logs.push("server started".to_string());

    println!("{:?}", db.logger.logs);
}
```

Solution

```
[server started, server started]
→ rust-project cargo run
  Compiling scheduler v0.1.0 (/Users/harkirat/Projects/rust-project)
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.16s
  Running `target/debug/scheduler`
["server started", "db started"]
```

```
0 implementations
struct Logger {
    logs: Vec<String>,
}

0 implementations
struct Database {
    logger: Rc<RefCell<Logger>>,
}

0 implementations
struct Server {
    logger: Rc<RefCell<Logger>>,
}

► Run | ⌘ Debug
fn main() {
    let logger: Rc<RefCell<Logger>> = Rc::new(RefCell::new(Logger { logs: vec![] }));

    let db: Database = Database { logger: logger.clone() };

    let server: Server = Server { logger: logger.clone() }; // use of moved value

    server.logger.borrow_mut().logs.push("server started".to_string());
    db.logger.borrow_mut().logs.push("db started".to_string());

    println!("{:?}", db.logger.borrow().logs);
}
```


Q1 - Arent there two mutable
borrows here?
Shouldnt that panic at runtime?

```
[server started, server started]  
→ rust-project cargo run  
Compiling scheduler v0.1.0 (/Users/harkirat/Projects/rust-project)  
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.16s  
Running `target/debug/scheduler`  
["server started", "db started"]
```

```
0 implementations  
struct Logger {  
    logs: Vec<String>,  
}  
  
0 implementations  
struct Database {  
    logger: Rc<RefCell<Logger>>,  
}  
  
0 implementations  
struct Server {  
    logger: Rc<RefCell<Logger>>,  
}  
  
► Run | ⚙ Debug  
fn main() {  
    let logger: Rc<RefCell<Logger>> = Rc::new(RefCell::new(Logger { logs: vec![] }));  
  
    let db: Database = Database { logger: logger.clone() };  
  
    let server: Server = Server { logger: logger.clone() }; // use of moved value  
  
    server.logger.borrow_mut().logs.push("server started".to_string());  
    db.logger.borrow_mut().logs.push("db started".to_string());  
  
    println!("{:?}", db.logger.borrow().logs);  
}
```

Q1 - Arent there two mutable borrows here?

Shouldnt that panic at runtime?

Ans - First mutable borrow drops right after the line

```
[server started, server started]
→ rust-project cargo run
  Compiling scheduler v0.1.0 (/Users/harkirat/Projects/rust-project)
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.16s
  Running `target/debug/scheduler`
["server started", "db started"]
```

```
0 implementations
struct Logger {
    logs: Vec<String>,
}

0 implementations
struct Database {
    logger: Rc<RefCell<Logger>>,
}

0 implementations
struct Server {
    logger: Rc<RefCell<Logger>>,
}

► Run | ⚙ Debug
fn main() {
    let logger: Rc<RefCell<Logger>> = Rc::new(RefCell::new(Logger { logs: vec![] }));

    let db: Database = Database { logger: logger.clone() };

    let server: Server = Server { logger: logger.clone() }; // use of moved value

    server.logger.borrow_mut().logs.push("server started".to_string());
    db.logger.borrow_mut().logs.push("db started".to_string());

    println!("{:?}", db.logger.borrow().logs);
}
```


Q2 - Why both Rc and Refcell are needed?

```
0 implementations
struct Logger {
    logs: Vec<String>,
}

0 implementations
struct Database {
    logger: Rc<RefCell<Logger>>,
}

0 implementations
struct Server {
    logger: Rc<RefCell<Logger>>,
}

► Run | ⚙ Debug
fn main() {
    let logger: Rc<RefCell<Logger>> = Rc::new(RefCell::new(Logger { logs: vec![] }));

    let db: Database = Database { logger: logger.clone() };

    let server: Server = Server { logger: logger.clone() }; // use of moved value

    server.logger.borrow_mut().logs.push("server started".to_string());
    db.logger.borrow_mut().logs.push("db started".to_string());

    println!("{:?}", db.logger.borrow().logs);
}
```


Q2 - Why both Rc and Refcell are needed?

Rc is for multiple owners

Refcell is for interior mutability

```
0 implementations
struct Logger {
    logs: Vec<String>,
}

0 implementations
struct Database {
    logger: Rc<RefCell<Logger>>,
}

0 implementations
struct Server {
    logger: Rc<RefCell<Logger>>,
}

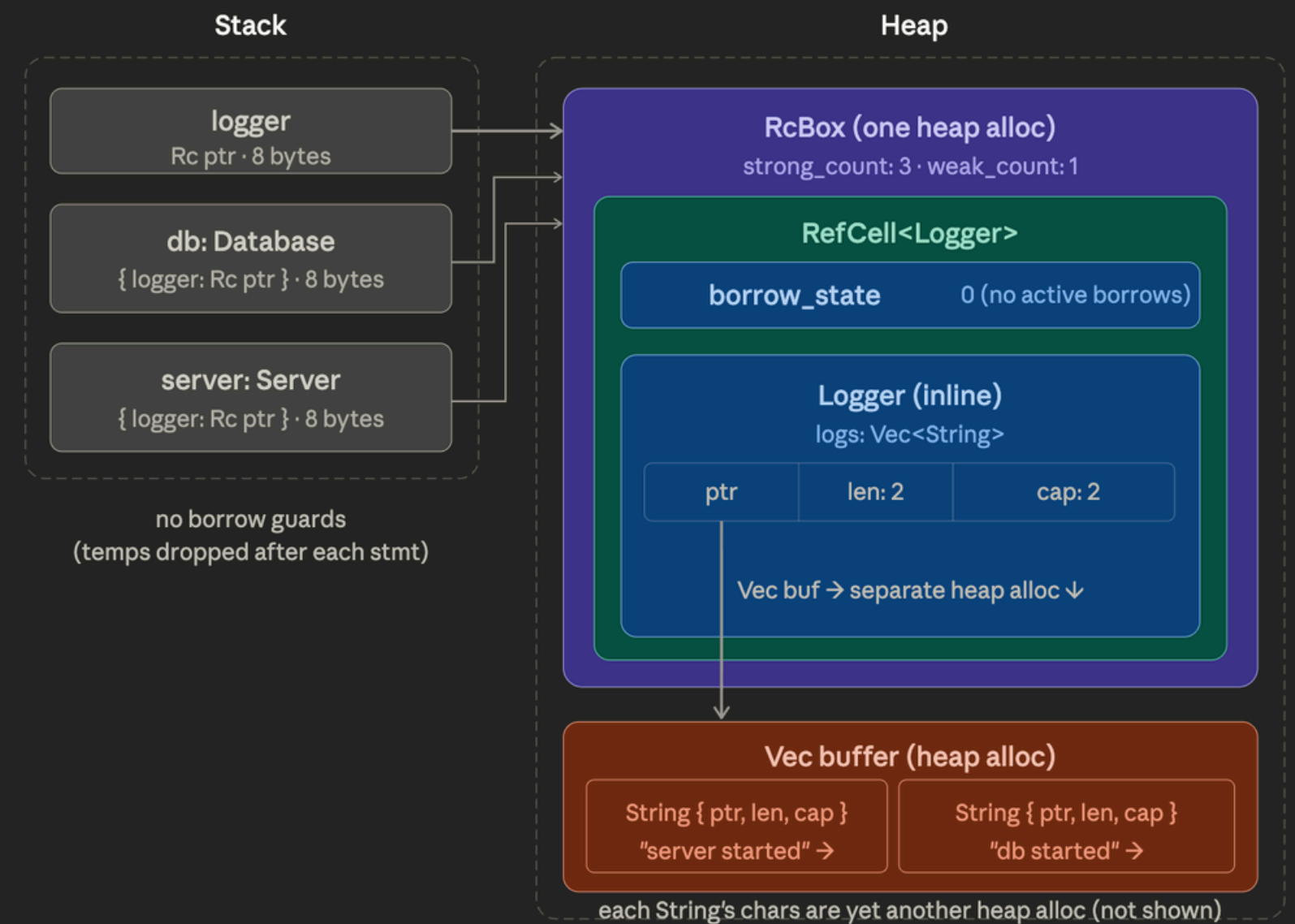
► Run | ⌘ Debug
fn main() {
    let logger: Rc<RefCell<Logger>> = Rc::new(RefCell::new(Logger { logs: vec![] }));

    let db: Database = Database { logger: logger.clone() };

    let server: Server = Server { logger: logger.clone() }; // use of moved value

    server.logger.borrow_mut().logs.push("server started".to_string());
    db.logger.borrow_mut().logs.push("db started".to_string());

    println!("{:?}", db.logger.borrow().logs);
}
```



Topics

1. Box
2. Deref
3. Rc
4. RefCell
5. Arc
6. Mutex
7. mpsc

Multithreaded memory
management

Super 30 2.0 has a
live class on it that
we will post